GPU 生态

GPU vs CPU 详细对比表格

表1：核心技术特性对比

对比维度	CPU	GPU	优势方
核心数量	8-64核（高端服务器）	2,000-18,000核（CUDA核心）	GPU（100倍+）
时钟频率	3.0-5.0 GHz	1.2-2.5 GHz	CPU
内存带宽	100-200 GB/s（DDR5）	600-2000 GB/s（GDDR6/HBM2）	GPU（10倍）
浮点性能	1-2 TFLOPS（单精度） （双精度）	20-80 TFLOPS（单精度） 10-20 TFLOPS（双精度）	GPU（10-40倍）
功耗	150-350W	250-700W	CPU
能效比	0.3-0.5 GFLOPS/W	30-100 GFLOPS/W	GPU（100倍+）
并行模型	SIMD（单指令多数据）	SIMT（单指令多线程）	GPU
适合情形	复杂逻辑、分支多	大规模并行、数据密集	取决于任务

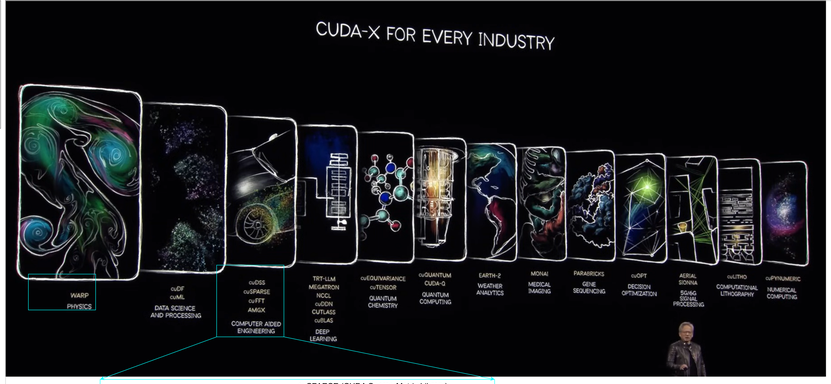
CAE仿真性能对比（实测数据）

仿真类型	CPU基准	GPU加速比	典型GPU	备注
粒子仿真（DEM）	1x	8-100倍	RTX 3060	千万级粒子
有限元（显式）	1x	50-200倍	RTX 4090	碰撞分析
有限元（隐式）	1x	5-20倍	A100	求解线性系统
CFD传统（FVM）	1x	50-200倍	A100	Navier-Stokes
CFD-LBM	1x	250-2700倍	4×A100	格子玻尔兹曼
流体（SPH）	1x	100-500倍	RTX 4090	光滑粒子
分子动力学	1x	150-800倍	A100	GROMACS测试

GPU计算底层软件平台全景图

主要GPU编程平台对比

平台	厂商	开放性	生态成熟度	主要应用
CUDA	NVIDIA	❌ 封闭	★★★★★	AI、HPC、科学计算，15年技术积累 → 垂直整合 → 生态锁定，成为事实的操作系统
OpenCL	Khronos Group	✅ 开放	★★	跨平台计算，OpenCL相同算法，60-80% GPU性能•没有开或生态•开发体验差
ROCm	AMD	✅ 开放	★★★	AI、HPC（追赶中）
oneAPI	Intel	✅ 开放	★★	异构计算
Metal	Apple	❌ 封闭	★★★	macOS/iOS GPU
SYCL	Khronos Group	✅ 开放	★	C++异构编程
Vulkan Compute	Khronos Group	✅ 开放	★★	游戏+计算



cuDSS (CUDA Direct Sparse Solver)
功能：稀疏矩阵直接求解器
应用：
结构分析（应力、应变计算）
电磁场仿真
流体动力学求解
优势：GPU加速的LU分解和稀疏线性方程组求解

cuSPARSE (CUDA Sparse Matrix Library)
功能：稀疏矩阵运算库
核心操作：
稀疏矩阵-向量乘法（SpMV）
稀疏矩阵-矩阵乘法（SpMM）
矩阵格式转换（CSR、COO、CSC等）
为什么重要：工程问题中90%以上的矩阵是稀疏的

cuFFT (CUDA Fast Fourier Transform)
功能：快速傅里叶变换库应用场景：
信号处理图像处理频谱分析声学仿真
性能：比CPU版本快10-50倍

AMGX (Algebraic Multigrid Solver)
功能：代数多重网格求解器
特点：专门处理超大规模稀疏线性系统迭代求解器，
适合并行计算比传统方法快数百倍
典型应用：CFD（计算流体力学）结构力学油藏模拟

厂商产品GPU加速状态采用的CUDA库

ANSYS Mechanical/Fluent/Maxwell ✅ 部分支持cuSPARSE, AMGX, cuSOLVER
Altair OptiStruct/AcuSolve ✅ 积极支持多个CUDA-X库
SiemensSimcenter (Star-CCM+) ✅ 较好支持自研+CUDA混合
DassaultAbaqus/SIMULIA ⚠️ 有限支持部分模块**MSC Software**Nastran/Adams ⚠️ 缓慢推进选择性采用

传统CAE厂商正在"被迫加速"拥抱GPU，但这是一场**十年级别的迁移工程**，而非一蹴而就。

Warp 介绍

Differentiable Spatial Computing for Python

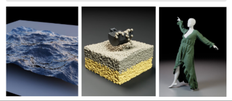
- CUDA is general purpose, efficient, but device centric:
  - C++ focused
  - Not specialized for simulation
  - Build everything yourself
  - Not differentiable
- Python is the lingua franca for AI
  - Low barrier to entry
  - Fast iteration and deployment
  - Rich ecosystem of DL frameworks
  - Often has poor performance
- How can we bridge the gap between CUDA and Python for simulation developers?

NVIDIA + python

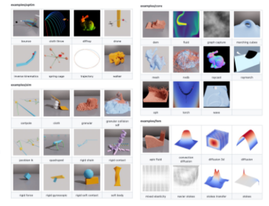
```
pip install warp-lang
```

1. GPU Kernels in Python
   - Write CUDA kernels in 100% Python syntax
   - Runtime JIT compilation
   - Fast developer iteration
2. Spatial Computing
   - Rich vector math library
   - Mesh processing and queries
   - Sparse volumes (OpenVDB)
   - Hash grids
3. Differentiable Programming
   - Auto-differentiation
   - Forward + backward kernel generation
   - Interop with DL frameworks (e.g.: PyTorch, JAX)
4. Omniverse Integration
   - OpenUSD import / export
   - Runtime extensions for Isaac / Composer

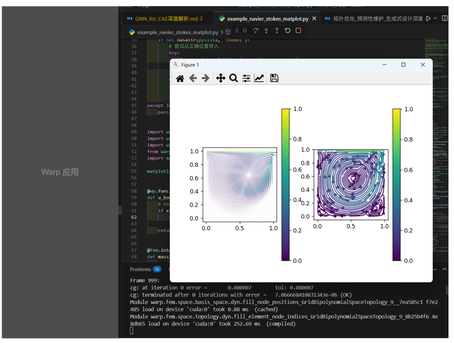
```
import warp as wp

@wp.kernel
def integrate(x: wp.array(dtype=wp.vec3),
              v: wp.array(dtype=wp.vec3),
              f: wp.array(dtype=wp.vec3),
              m: wp.array(dtype=float)):

    # thread id
    tid = wp.tid()

    # Semi-implicit Euler step
    v[tid] = v[tid] + f[tid] * m[tid] * wp.vec3(0.0, -9.8, 0.0) * dt
    x[tid] = x[tid] + v[tid] * dt

# kernel launch
wp.launch(integrate, dim=1024, inputs=[x, v, f, _], device="cuda:0")
```

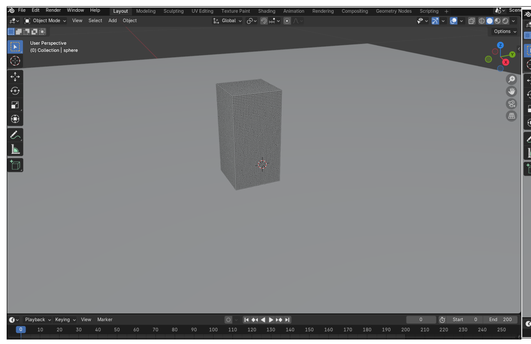


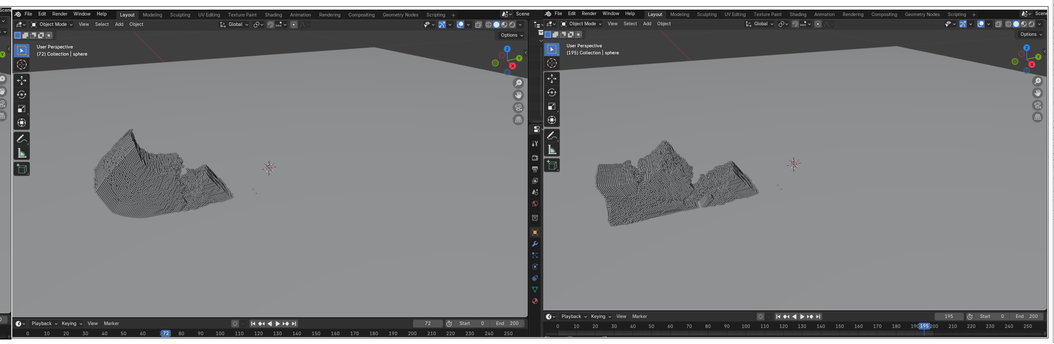
- warp.core
  - Differentiable kernel coding for Python
  - Math, geometry, vector library
- warp.sim
  - Differentiable real-time simulation for robotic control + prediction
  - Rigid bodies, soft bodies, particles, cloth
  - URDF, MJCF, UsdPhysics parsers
- warp.fem (early access)
  - Differentiable PDE framework for heat transfer, diffusion, elasticity
  - Fast iteration, but offline focus
  - Not replacing existing codes, but potential to build on them
- warp.llm (early access)
  - AI agent specialized for Warp kernel coding
  - Generate simulation code directly from prompts
  - Generate loss functions from code -> optimization

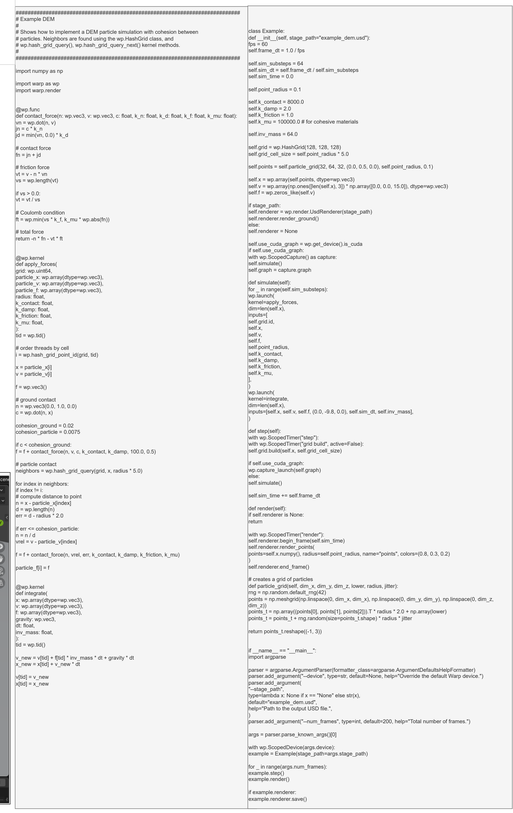


Warp 应用





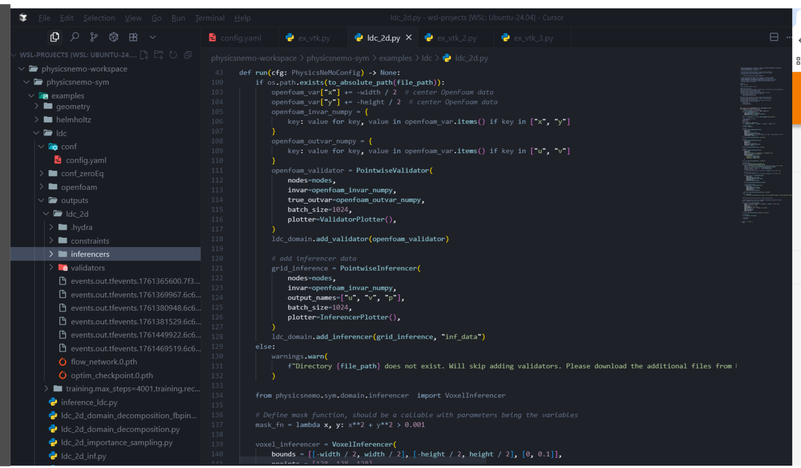
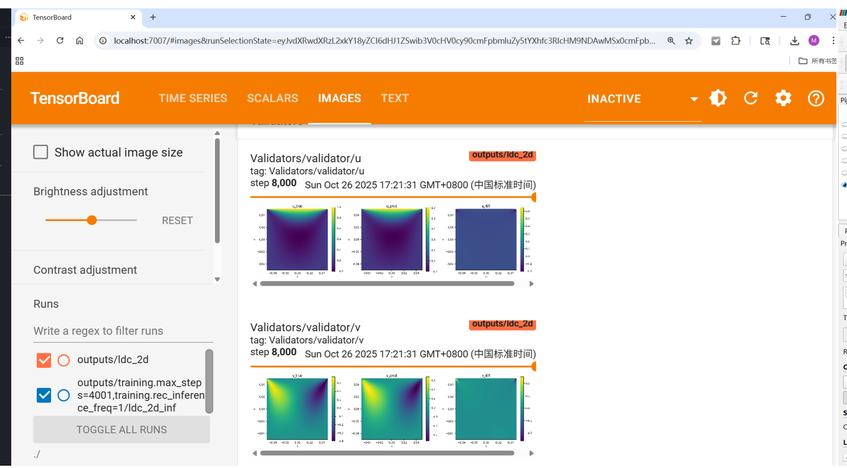
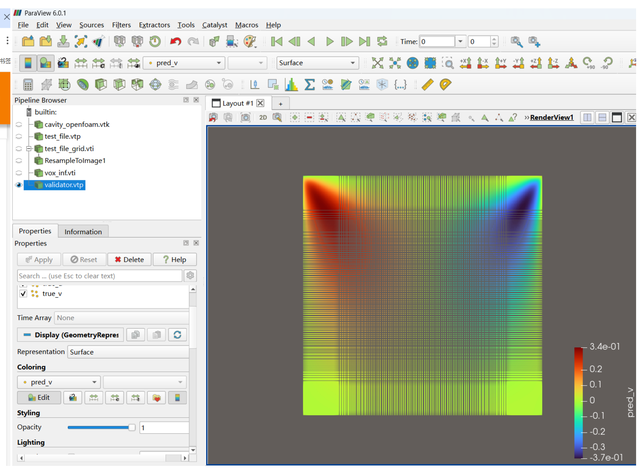




Warp vs. JAX

"✓ GPU加速": "支持CUDA",
"✓ 自动微分": "梯度优化",
"✓ JIT编译": "提升性能",
"✓ 科学计算": "都能写数值仿真",
"✓ Python接口": "科学家友好"

```
科学计算与物理AI生态
├── 【空间计算派】
│   NVIDIA Warp
│   ├ 粒子系统
│   ├ 几何操作
│   ├ 碰撞检测
│   ├ 实时渲染
└── 【张量计算派】
    Google JAX
    ├ 神经网络
    ├ 张量运算
    ├ 谱方法
    ├ 批处理
```

您的主要需求	首选	次选	组合方式
机器人仿真	Warp ★★★★★	-	纯Warp
拓扑优化	JAX ★★★★★	Warp FEM	JAX优化+Warp求解
CFD（网格法）	Warp ★★★★	JAX	Warp主+JAX辅
CFD（LBM）	JAX-XLB ★★★★★	Warp	XLB仿真+Warp可视化
预测性维护	Warp+JAX ★★★★★	-	Warp孪生+JAX预测
生成式设计	JAX ★★★★★	-	纯JAX
粒子系统	Warp ★★★★★	JAX-MD	看稀疏vs稠密

Physicsnemo